

MACHINE LEARNING

Units 3, 4 & 5 — Complete Exam Answer Key

SRK Institute of Technology | B.Tech CSE | Based on Prescribed Textbook

Contents: Unit 3 (Q1–Q4) | Unit 4 (Q1–Q8) | Unit 5 Clustering (Q1–Q8) | Short Answer (Q1–Q10)

UNIT 3 — Bayesian Methods & Decision Trees

U3-Q1. Bayes Rule + Cancer Test Problem (5M)

Statement of Bayes' Rule

Bayes' Rule converts a prior probability into a posterior probability using observed evidence. For events A (disease) and B (test result):

$$P(A|B) = P(B|A) \times P(A) / [P(B|A) \times P(A) + P(B|A^c) \times P(A^c)]$$

This is: Posterior = (Likelihood × Prior) / Evidence.

Given

- True Positive Rate (Sensitivity): $P(\text{Test+}|\text{Cancer}) = 0.85$
- True Negative Rate (Specificity): $P(\text{Test-}|\text{No Cancer}) = 0.90$
- False Positive Rate: $P(\text{Test+}|\text{No Cancer}) = 1 - 0.90 = 0.10$
- Prevalence: 10 out of 300 → $P(\text{Cancer}) = 10/300 = 1/30 \approx 0.0333$
- $P(\text{No Cancer}) = 290/300 \approx 0.9667$

Applying Bayes' Rule

$$\begin{aligned} P(\text{Cancer}|\text{Test+}) &= P(\text{Test+}|\text{Cancer}) \times P(\text{Cancer}) / [P(\text{Test+}|\text{Cancer}) \times P(\text{Cancer}) + P(\text{Test+}|\text{No Cancer}) \times P(\text{No Cancer})] \\ &= (0.85 \times 10/300) / [(0.85 \times 10/300) + (0.10 \times 290/300)] \\ &= (0.85 \times 10) / [(0.85 \times 10) + (0.10 \times 290)] \\ &= 8.5 / (8.5 + 29) = 8.5 / 37.5 \approx 0.2267 \rightarrow 22.67\% \end{aligned}$$

Event	Calculation	Value
P(Cancer)	10/300	0.0333
P(No Cancer)	290/300	0.9667
P(Test+ Cancer)	Given (TPR)	0.85

P(Test+ No Cancer)	1-TNR=1-0.90	0.10
P(Cancer Test+)	8.5/37.5	≈ 0.2267 (22.67%)

Result: Even with 85% sensitivity, only ~22.67% of those who test positive truly have cancer. This counterintuitive result occurs because the disease prevalence is very low (10/300). The large pool of healthy people generates many false positives, dominating the test-positive group.

U3-Q2. Random Forest — Characteristics and Algorithm (5M)

Introduction

Random Forest (RF) is a combinational ML model that employs a collection of randomly generated Decision Trees (DTs) for prediction. It was introduced to overcome the expensive and often suboptimal nature of a single DT on high-dimensional data.

Characteristics of Random Forest (from Textbook Section 3.8)

- It is a COLLECTION of randomly generated DTs — hence the name 'forest'.
- Each DT makes a prediction and the overall result is obtained by: (a) Majority voting in classification, (b) Average of all DT predictions in regression.
- The underlying phenomenon is called BAGGING (Bootstrap Aggregation).
- DTs can be built in PARALLEL — each tree's output is computed independently. Only the final aggregation requires centralised processing.
- Model performance improves if each DT works well AND the correlation among DTs is small.
- It can deal with very large-dimensional data sets.
- It REDUCES VARIANCE by: (1) using different samples of data in building DTs to reduce correlation between trees, and (2) simplifying each DT by using only a random subset of features.

Bootstrapping Process

Given n training patterns in an L -dimensional space, RF works as follows:

1. Select m samples, each of size n , with REPLACEMENT (same data point can appear more than once in a sample).
2. In each sample, select a random subset of $l < L$ features (typically $l = \sqrt{L}$). So each sample is an $n \times l$ data matrix.
3. Construct a DT for each sample using only l features. Even though there are L features total, each tree uses only l — high-dimensional data can be handled.
4. For a test pattern x , compute outputs of all m DTs and AGGREGATE: majority vote (classification) or average (regression).

RF Classification Algorithm

5. Bootstrap: Create m training sets $D_1, D_2, \dots, D_m$ by sampling n points with replacement.
6. Feature Subset: For each D_i , randomly select $l < L$ features.
7. Build DT: Train a DT on each $(D_i, l\text{-features})$ pair independently.
8. Predict: For test x , get prediction y_i from each tree T_i .
9. Aggregate: Final class = argmax of majority votes across all m trees.

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \dots, T_m(x))$$

RF Regression Algorithm

10. Steps 1–4 are the same as classification.

11. Aggregate: Final prediction = average of all tree outputs.

$$\hat{y} = (1/m) \times \sum_i T_i(x)$$

Performance on Olivetti Face Data (from Textbook)

From Fig. 3.15 in the textbook: On the Olivetti Face dataset (360 training, 40 test images):

- Single DTC with optimal α : accuracy ≈ 0.595 (Gini) / 0.547 (entropy).
- RF with 500 DTs: maximum accuracy ≈ 0.967 .
- RF with 100 DTs consistently outperforms 10 or 50 DTs.

Property	Single DT	Random Forest
Training	Can overfit	Bootstrapping reduces overfit
Variance	High	Low (averaging reduces variance)
Bias	Low to High	Low
Speed	Fast	Parallel — scalable
High-dim Data	Greedy, may fail	Handles well via feature subsets
Interpretability	High (rules)	Low (black box)

U3-Q3. Class Conditional Independence & Naïve Bayes Classifier (5M)

Class Conditional Independence — Definition

Class-conditional independence is the assumption that each feature is independent of every other feature given the class label. Formally, for features $f_1, f_2, \dots, f_m$ and class C :

$$P(f_1=v_1, f_2=v_2, \dots, f_m=v_m | C) = \prod_j P(f_j=v_j | C)$$

This simplifies computation exponentially. The resulting classifier is the Naïve Bayes Classifier (NBC). From the textbook: 'NBC employs class-conditional independence. It is given as:

$$P(f_1=v_1, \dots, f_m=v_m | C) = P(f_1=v_1 | C) \times P(f_2=v_2 | C) \times \dots \times P(f_m=v_m | C)'$$

Temperature (Play Tennis) Dataset — 14 Training Instances

#	Outlook	Temp	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes

6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Prior Probabilities:

$$P(\text{Yes}) = 9/14, \quad P(\text{No}) = 5/14$$

Frequency Tables (Conditional Probabilities)

Feature	Value	P(Yes)	P(No)
Outlook	Sunny	2/9	3/5
	Overcast	4/9	0/5
	Rain	3/9	2/5
Temperature	Hot	2/9	2/5
	Mild	4/9	2/5
	Cool	3/9	1/5
Humidity	High	3/9	4/5
	Normal	6/9	1/5
Wind	Weak	6/9	2/5
	Strong	3/9	3/5

Test Data 1 = (Rain, Cool, Normal, Weak)

$$\begin{aligned}
P(\text{Yes}|\mathbf{x}_1) &\propto P(\text{Yes}) \times P(\text{Rain}|\text{Yes}) \times P(\text{Cool}|\text{Yes}) \times P(\text{Normal}|\text{Yes}) \times P(\text{Weak}|\text{Yes}) \\
&= (9/14) \times (3/9) \times (3/9) \times (6/9) \times (6/9) \\
&= (9/14) \times (1/3) \times (1/3) \times (2/3) \times (2/3) \\
&= (9/14) \times (4/81) = 36/1134 \approx 0.03175 \\
P(\text{No}|\mathbf{x}_1) &\propto (5/14) \times (2/5) \times (1/5) \times (1/5) \times (2/5) \\
&= (5/14) \times (4/625) = 20/8750 \approx 0.00229
\end{aligned}$$

Since $0.03175 > 0.00229 \rightarrow \text{Test Data 1} = (\text{Rain, Cool, Normal, Weak}) \rightarrow \text{CLASS: PLAY} = \text{YES}$
✓

Test Data 2 = (Sunny, Mild, High, Weak)

$$\begin{aligned}P(\text{Yes}|\mathbf{x}_2) &\propto (9/14) \times P(\text{Sunny}|\text{Yes}) \times P(\text{Mild}|\text{Yes}) \times P(\text{High}|\text{Yes}) \times P(\text{Weak}|\text{Yes}) \\&= (9/14) \times (2/9) \times (4/9) \times (3/9) \times (6/9) \\&= (9/14) \times (144/6561) = 1296/91854 \approx 0.01411 \\P(\text{No}|\mathbf{x}_2) &\propto (5/14) \times (3/5) \times (2/5) \times (4/5) \times (2/5) \\&= (5/14) \times (48/625) = 240/8750 \approx 0.02743\end{aligned}$$

Since $0.02743 > 0.01411 \rightarrow \text{Test Data 2} = (\text{Sunny, Mild, High, Weak}) \rightarrow \text{CLASS: PLAY} = \text{NO} \checkmark$

U3-Q4. Bias-Variance Trade-Off (5M)

Definitions

Bias:

Error from approximating a real-world problem with a simplified model. High-bias models UNDERFIT — they fail to capture the underlying pattern even on training data.

Variance:

Error from sensitivity to small fluctuations in training data. High-variance models OVERFIT — they fit training data perfectly but fail to generalise.

Key Principle (from Textbook Section 3.7): Simple models tend to have higher bias while complex models tend to have higher variance. One needs to learn a model that strikes a balance. This phenomenon is called the Bias–Variance Trade-off.

Textbook Example 7 (Table 3.7)

Dataset: 4 data points. First 3 used for training, 4th ($x=1/2$, true $y=7/4$) for testing.

Example	x	Target y	y_sl (Linear)	y_p1 (Poly-3a)	y_p2 (Poly-3b)
1	0	1	2/3	1	1
2	1	3	11/3	3	3
3	2	7	20/3	7	7
4 (Test)	1/2	7/4	2.16	1.84	1.87

The true polynomial is $y = 1 + x + x^2$ (degree-2), which fits the 4th row exactly (predicts $7/4 = 1.75$).

Analysis

Linear Model ($y_{sl} = 3x + 2/3$):

- Makes errors even on training data ($2/3 \neq 1$, $11/3 \neq 3$, etc.)
- UNDERFITS — too simple to capture quadratic structure
- HIGH BIAS, LOW VARIANCE (unique prediction on test point)

Degree-3 Polynomials (y_{p1} , y_{p2}):

- Zero training error — fit all 3 training points perfectly

- OVERFIT — infinitely many degree-3 polynomials fit 3 points
- Different test predictions: 1.84 vs 1.87 — non-zero variance
- LOW BIAS, HIGH VARIANCE

Bias-Variance in Decision Trees

Property	Shallow/Pruned DT	Deep/Full DT
Complexity	Low	High
Bias	HIGH (underfits)	LOW
Variance	LOW	HIGH (overfits)
Training Error	High	Low/Zero
Test Error	Moderate	High
Solution	Increase depth	Pruning, cross-validation

From Fig. 3.13 in textbook: DT pruned to level 2 (Fig 3.13a) underfits — higher bias. DT with leaf nodes at level 4 (Fig 3.13b) overfits — higher variance. The balanced DT in Fig. 3.12 achieves best test performance.

Conclusion: Pruning Decision Trees is the primary mechanism to manage bias-variance trade-off — it reduces variance. The optimal model is obtained by tuning parameters (levels, nodes) to acceptable validation performance. This is the most important issue in ML model building.

UNIT 4 — Linear Discriminants, SVM, MLP

U4-Q1. AND Gate using Perceptron — LDF and Augmented Weight Vector (5M)

Linear Discriminant Function

A linear discriminant function separates two classes using a hyperplane. The general form is:

$$g(\mathbf{w}, \mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = \sum_i \mathbf{w}(i) \mathbf{x}(i) + b$$

where $\mathbf{w}$ is the weight vector, $\mathbf{x}$ is the pattern, and b is the bias threshold. Decision: if $g(\mathbf{x}) > 0 \rightarrow$ positive class; if $g(\mathbf{x}) < 0 \rightarrow$ negative class.

Augmented Weight Vector ($\mathbf{W}_a$)

The bias is absorbed into the weight vector by augmenting both vectors:

$$\mathbf{x}_a = [1, \mathbf{x}] \quad (\text{prepend } 1)$$

$$\mathbf{w}_a = [b, \mathbf{w}] \quad (\text{prepend bias})$$

$$g(\mathbf{w}_a, \mathbf{x}_a) = \mathbf{w}_a^T \mathbf{x}_a = b + \mathbf{w}^{(1)} \mathbf{x}^{(1)} + \mathbf{w}^{(2)} \mathbf{x}^{(2)}$$

AND Gate Truth Table & Augmented Patterns

A	B	Y=A·B	Label l_j	Augmented $x_a = (1,A,B)$
0	0	0	-1	(1,0,0)
0	1	0	-1	(1,0,1)
1	0	0	-1	(1,1,0)
1	1	1	+1	(1,1,1)

Perceptron Learning Algorithm — Step-by-Step

Start with $w_0 = (0,0,0)$. Pattern x_j is MISCLASSIFIED if $l_j \times (w_i^T x_j) \leq 0$.

Update: $w_{i+1} = w_i + l_j \times x_j$

Step	w_i	Pattern	$w_i^T x_j$	l_j	Correct?	Update $\rightarrow w_{i+1}$
0	(0,0,0)	(1,0,0)	0	-1	No	$w_1 = (0,0,0) + (-1)(1,0,0) = (-1,0,0)$
1	(-1,0,0)	(1,0,1)	-1	-1	Yes ✓	No update
2	(-1,0,0)	(1,1,0)	-1	-1	Yes ✓	No update
3	(-1,0,0)	(1,1,1)	-1	+1	No	$w_2 = (-1,0,0) + (+1)(1,1,1) = (0,1,1)$
4	(0,1,1)	(1,0,0)	0	-1	No	$w_3 = (0,1,1) + (-1)(1,0,0) = (-1,1,1)$
5	(-1,1,1)	(1,0,1)	0	-1	No	$w_4 = (-1,1,1) + (-1)(1,0,1) = (-2,1,0)$
6	(-2,1,0)	(1,1,0)	-1	-1	Yes ✓	No update
7	(-2,1,0)	(1,1,1)	-1	+1	No	$w_5 = (-2,1,0) + (1,1,1) = (-1,2,1)$
8	(-1,2,1)	(1,0,1)	0	-1	No	$w_6 = (-1,2,1) + (-1)(1,0,1) = (-2,2,0)$
9	(-2,2,0)	(1,1,0)	0	-1	No	$w_7 = (-2,2,0) + (-1)(1,1,0) = (-3,1,0)$
10	(-3,1,0)	(1,1,1)	-2	+1	No	$w_8 = (-3,1,0) + (1,1,1) = (-2,2,1)$
11	(-2,2,1)	(1,1,0)	0	-1	No	$w_9 = (-2,2,1) + (-1)(1,1,0) = (-3,1,1)$
12	(-3,1,1)	(1,1,1)	-1	+1	No	$w_{10} = (-3,1,1) + (1,1,1) = (-2,2,2)$
13	(-2,2,2)	(1,0,1)	0	-1	No	$w_{11} = (-2,2,2) + (-1)(1,0,1) = (-3,2,1)$
14	(-3,2,1)	(1,1,1)	0	+1	No	$w_{12} = (-3,2,1) + (1,1,1) = (-2,3,2)$
15	(-2,3,2)	(1,0,1)	0	-1	No	$w_{13} = (-2,3,2) + (-1)(1,0,1) = (-3,3,1)$
16	(-3,3,1)	(1,1,0)	0	-1	No	$w_{14} = (-3,3,1) + (-1)(1,1,0) = (-4,2,1)$
17	(-4,2,1)	(1,1,1)	-1	+1	No	$w_{15} = (-4,2,1) + (1,1,1) = (-3,3,2)$
18	(-3,3,2)	(1,1,0)	0	-1	No	$w_{16} = (-3,3,2) + (-1)(1,1,0) = (-4,2,2)$
19	(-4,2,2)	(1,1,1)	0	+1	No	$w_{17} = (-4,2,2) + (1,1,1) = (-3,3,3)$
20	(-3,3,3)	(1,0,1)	0	-1	No	$w_{18} = (-3,3,3) + (-1)(1,0,1) = (-4,3,2)$
21	(-4,3,2)	ALL OK	-	-	ALL ✓	TERMINATE

Final Augmented Weight Vector: $W_a = (-4, 3, 2) \rightarrow b = -4, w(A) = 3, w(B) = 2$

Linear Discriminant Function for AND Gate

$$g(\mathbf{x}) = 3A + 2B - 4$$

Verification:

- (0,0): $0+0-4 = -4 < 0 \rightarrow$ Class 0 (No AND) ✓
- (0,1): $0+2-4 = -2 < 0 \rightarrow$ Class 0 ✓
- (1,0): $3+0-4 = -1 < 0 \rightarrow$ Class 0 ✓
- (1,1): $3+2-4 = +1 > 0 \rightarrow$ Class 1 (AND=1) ✓

Decision Boundary: $3A + 2B - 4 = 0$ (a line separating AND=1 from AND=0 in 2D feature space).

U4-Q2/Q6. SVM — Objectives, Hard Margin & Soft Margin Formulation (5M+5M)

Introduction

When classes are linearly separable, infinitely many decision boundaries exist. Perceptron finds one arbitrarily. SVM finds the OPTIMAL boundary that maximises the margin of separation between the two classes (Section 8.4).

SVM Objectives

- Find the decision boundary $w^T x + b = 0$ that is evenly centred between the two classes.
- Maximise the margin $2/||w||$ — the distance between the two support planes.
- Achieve GLOBAL OPTIMAL SOLUTION through a well-formulated convex optimization problem.
- Extend to non-linear boundaries using the Kernel Trick.
- Can be used as a feature selector (weights with larger absolute value = more important features).

Hard Margin Formulation (Section 8.4)

Setup:

Training set: $\{(x_i, y_i)\}$, $i=1, \dots, n$ where $y_i \in \{-1, +1\}$. $y_i=-1$: negative class; $y_i=+1$: positive class.

Constraints:

$$\begin{aligned} w^T x_i + b &\geq +\epsilon && \text{for all } i \text{ with } y_i=+1 \\ w^T x_i + b &\leq -\epsilon && \text{for all } i \text{ with } y_i=-1 \end{aligned}$$

Dividing by ϵ (let $w'=w/\epsilon$, $b'=b/\epsilon$):

$$\begin{aligned} w'^T x_i + b' &\geq +1 && \text{for } y_i=+1 \quad (\text{positive support plane}) \\ w'^T x_i + b' &\leq -1 && \text{for } y_i=-1 \quad (\text{negative support plane}) \end{aligned}$$

Combined: $y_i(w^T x_i + b) \geq 1, \forall i$

Points on the support planes (equality holds) are SUPPORT VECTORS.

Margin = total distance between two support planes:

$$\text{Margin} = 2/||w||$$

Optimization Problem:

$$\begin{aligned} \text{Minimize:} \quad & (1/2) ||\mathbf{w}||^2 \\ \text{Subject to:} \quad & \mathbf{y}_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i = 1, \dots, n \end{aligned}$$

Lagrangian:

$$L = (1/2) \mathbf{w}^T \mathbf{w} + \sum_i \alpha_i (1 - \mathbf{y}_i (\mathbf{w}^T \mathbf{x}_i + b))$$

KKT Conditions:

$$\begin{aligned} \mathbf{w} &= \sum_i \alpha_i \mathbf{y}_i \mathbf{x}_i \quad (\text{Support vectors contribute to } \mathbf{w}) \\ \sum_i \alpha_i \mathbf{y}_i &= 0 \\ \alpha_i &\geq 0 \quad \text{and} \quad \alpha_i (1 - \mathbf{y}_i (\mathbf{w}^T \mathbf{x}_i + b)) = 0 \end{aligned}$$

Only support vectors (on the margin planes) have $\alpha_i > 0$. Well-classified points far from boundary have $\alpha_i = 0$ and do NOT contribute to $\mathbf{w}$.

Textbook Example 10 (Hard Margin)

Negative class: (2,1) and (1,3). Positive class: (6,3). System of 3 equations:

$$\begin{aligned} 2\mathbf{w}(1) + \mathbf{w}(2) + b &= -1, & \mathbf{w}(1) + 3\mathbf{w}(2) + b &= -1, & 6\mathbf{w}(1) + 3\mathbf{w}(2) + b &= +1 \\ \text{Solution: } \mathbf{w}(1) &= 2/5, & \mathbf{w}(2) &= 1/5, & b &= -2 \end{aligned}$$

Support vectors: {(2,1), (6,3)}. Point (1,3) has $\alpha_2 = 0$ (not a support vector).

Soft Margin Formulation (Section 8.4.1)

Hard margin fails when data is NOT perfectly linearly separable. Soft margin introduces slack variables $\xi_i \geq 0$ to allow some margin violations:

$$\begin{aligned} \text{Minimize:} \quad & (1/2) ||\mathbf{w}||^2 + C \cdot \sum_i \xi_i \\ \text{Subject to:} \quad & \mathbf{y}_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad -\xi_i \leq 0, \quad \forall i \end{aligned}$$

C is the regularization parameter:

- $C \rightarrow \infty$: soft-margin collapses to hard-margin (no violations allowed).
- $C \rightarrow 0$: any amount of violation is tolerated.
- Need to choose appropriate C for good performance.

Lagrangian for Soft Margin:

$$L = (1/2) \mathbf{w}^T \mathbf{w} + C \cdot \sum_i \xi_i + \sum_i \alpha_i (1 - \xi_i - \mathbf{y}_i (\mathbf{w}^T \mathbf{x}_i + b)) - \sum_i \lambda_i \xi_i$$

KKT Conditions:

$$\begin{aligned} \nabla_{\mathbf{w}} L &= 0 \rightarrow \mathbf{w} = \sum_i \alpha_i \mathbf{y}_i \mathbf{x}_i \\ \partial L / \partial b &= 0 \rightarrow \sum_i \alpha_i \mathbf{y}_i = 0 \\ \partial L / \partial \xi_i &= 0 \rightarrow \alpha_i + \lambda_i = C \rightarrow \alpha_i \in [0, C] \end{aligned}$$

Lagrange multipliers are bounded by C; this allows points to violate the margin to the extent quantified by ξ_i .

Property	Hard Margin SVM	Soft Margin SVM
Assumption	Linearly separable	Not perfectly separable
Slack variables	None	$\xi_i \geq 0$
α_i range	$\alpha_i \geq 0$	$\alpha_i \in [0, C]$
Parameter	None	C (regularization)
Violators	Not allowed	Allowed (penalised by $C \cdot \sum \xi_i$)

U4-Q3. Non-linear SVM and Kernel Trick (5M)

Problem with Linear SVM

When the data is NOT linearly separable in the input space (e.g., Boolean XOR), a linear decision boundary cannot correctly classify all patterns. Example: XOR cannot be represented as $ax(1)+bx(2)+c$ (shown in Table 8.6 of textbook).

Non-linear SVM Idea

Map data from the input space to a higher-dimensional FEATURE SPACE where it becomes linearly separable, then apply SVM in that space.

$$\phi: \mathbb{R}^1 \rightarrow \mathbb{R}^H \quad \text{where } H \gg 1$$

For XOR (Table 8.7): Adding $x(1)x(2)$ as a third variable creates a 3D space where XOR IS linearly separable. The linear form $x(1)+x(2)-2x(1)x(2)$ captures XOR in 3D.

Two Problems with Explicit Mapping

- The data may not be linearly separable even in the feature space $\rightarrow$ handled by soft-margin formulation.
- We may not know the explicit representation ϕ , and computation in feature space may be unwieldy even if known $\rightarrow$ handled by the Kernel Trick.

The Kernel Trick (Section 8.4.3)

The key insight: for SVM classification, we only need the INNER PRODUCT $\phi(x_i)^T \phi(x_j)$ in the feature space, not the explicit vectors. A kernel function K computes this inner product directly:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

So all computations happen in the LOW-DIMENSIONAL input space, even though conceptually we are working in a high (possibly infinite) dimensional feature space.

Classification rule becomes:

$$\mathbf{w}^T \phi(\mathbf{x}) + \mathbf{b} = \sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + \mathbf{b}$$

Popular Kernel Functions (from Textbook)

Kernel Name	Formula	Notes
Polynomial (degree p)	$K(x_i, x) = (x_i^T x)^p$	Polynomial mapping
Polynomial (up to degree p)	$K(x_i, x) = (1 + x_i^T x)^p$	Includes lower degrees
Gaussian (RBF)	$K(x_i, x) = \exp(-\ x_i - x\ ^2 / 2\sigma^2)$	Maps to ∞ -dim space; σ controls width
Sigmoidal	$K(x_i, x) = \tanh(ax_i^T x + b)$	Similar to neural network activation

Worked Example: Polynomial Kernel Degree 2

For 2D input $x_i = (x_i(1), x_i(2))$:

$$\begin{aligned} K(\mathbf{x}_i, \mathbf{x}) &= (\mathbf{x}_i^T \mathbf{x})^2 = (x_i(1)x(1) + x_i(2)x(2))^2 \\ &= x_i(1)^2 x(1)^2 + x_i(2)^2 x(2)^2 + 2x_i(1)x_i(2)x(1)x(2) \end{aligned}$$

This corresponds to feature mapping: $\phi(x_i) = (x_i(1)^2, x_i(2)^2, \sqrt{2} \cdot x_i(1)x_i(2))^T$

So using the degree-2 polynomial kernel is equivalent to mapping to 3D and computing the dot product there — without ever explicitly computing the 3D vectors.

Key Advantage: The Gaussian kernel maps to an INFINITE-dimensional space. Without the kernel trick, such computations would be impossible. With the kernel, everything is computed in the original low-dimensional input space.

U4-Q4/Q7. Linear Discriminants for Classification (5M)

Linear Discriminant Function (Section 8.2)

A linear discriminant function separates patterns into classes using a hyperplane. For an l -dimensional pattern $x = (x(1), \dots, x(l))$:

$$g(w, x) = w^T x + b = \sum_i w(i)x(i) + b$$

w is the weight vector (l -dimensional), b is the bias/threshold. Decision: assign x to positive class if $g(w, x) > 0$, else negative class.

- $g(w, x) = 0$ characterizes the DECISION BOUNDARY — a hyperplane in l -dimensional space.
- w is ORTHOGONAL to the decision boundary.
- The sign of b determines which side of the boundary the origin lies on.

Example (from Textbook Example 1)

For Boolean OR: $g(x) = x(1) + x(2) - 0.5 > 0$. Here $w = (1, 1)$, $b = -0.5$.

For Boolean AND: $g(x) = x(1) + x(2) - 1.5 > 0$. Here $w = (1, 1)$, $b = -1.5$.

Distance from Decision Boundary

$$r = g(x) / ||w||$$

Points in the positive half space have $g(x) > 0$ (angle between w and x is less than 90°). Points in the negative half space have $g(x) < 0$.

Linear Discriminants for Multi-class

For C classes $C_1, C_2, \dots, C_C$, define one discriminant per class:

$$g_i(x) = w_i^T x + b_i \quad \text{for } i=1, 2, \dots, C$$

Assign x to class C_i if $g_i(x) > g_j(x)$ for all $j \neq i$.

Example — Textbook Example 2

$w = (-1, 2)$, $b = 1$. Classify $x_1 = (1, 1)$ and $x_2 = (4, 1)$:

$$g(w, x_1) = -1 \times 1 + 2 \times 1 + 1 = 2 > 0 \rightarrow \text{Positive class } \checkmark$$

$$g(w, x_2) = -1 \times 4 + 2 \times 1 + 1 = -1 < 0 \rightarrow \text{Negative class } \checkmark$$

Perceptron as Linear Discriminant

Perceptron learns the weight vector w using training data. If classes are linearly separable, the Perceptron learning algorithm converges in finite steps to a valid w (Section 8.3).

SVM as Linear Discriminant

SVM chooses the unique OPTIMAL linear discriminant that maximises the margin. The optimal w is expressed as $w = \sum_{i \in S} \alpha_i y_i x_i$ (sum over support vectors only).

Summary Comparison

Model	Linear Discriminant	Learning Method	Decision Criterion
Perceptron	$w^T x + b$	Error-correction rule	Any valid boundary
SVM	$w^T x + b$	Constrained quadratic opt	Maximum margin boundary
Logistic Regression	$\sigma(w^T x + b)$	MLE / gradient descent	Probabilistic boundary
NBC (via LDF)	Log-likelihood ratio	Frequency counts	Bayes-optimal

U4-Q5. Perceptron — Structure and Learning Algorithm (5M)

Structure of Perceptron (Section 8.3)

The Perceptron is the earliest ML model and the building block for deep neural networks. It is a single artificial neuron.

Components:

- Inputs: $x(1), x(2), \dots, x(l)$ — feature values
- Weights: $w(1), w(2), \dots, w(l)$ — learned parameters
- Bias: b (or $w(0)$) — threshold
- Activation: $a = \sum_i w(i)x(i) + b$
- Output: $y = \text{sign}(a) \rightarrow +1$ if $a > 0$ (positive class), -1 if $a < 0$ (negative class)

Augmented Form:

$x_a = [1, x]$ and $w_a = [b, w]$. Discriminant: $g(w_a, x_a) = w_a^T x_a$

$$[1, x(1), x(2), \dots, x(l)] \times [b, w(1), w(2), \dots, w(l)]^T \rightarrow \text{sign}(\cdot) \rightarrow \text{output } y$$

Perceptron Learning Algorithm (Section 8.3.1)

Labels: $l_j \in \{-1, +1\}$. x_j is misclassified if $l_j \times (w_i^T x_j) \leq 0$.

12. Initialize: $w_0 = \text{zero vector}$, $i = 0$, $j = 1$.
13. If x_j is misclassified ($l_j \cdot w_i^T x_j \leq 0$): $i = i + 1$, $w_i = w_{i-1} + l_j \cdot x_j$. Set $j = (j+1) \bmod n$, go to Step 2.
14. Else: Set $j = (j+1) \bmod n$, go to Step 2.
15. TERMINATE if no change in w for n successive steps.

Update Rules:

If negative class misclassified ($l_j = -1$): $w_{i+1} = w_i - x_j$
If positive class misclassified ($l_j = +1$): $w_{i+1} = w_i + x_j$

Example — Boolean OR (Textbook Example 6)

Augmented patterns: $x_1 = (1, 0, 0), l_1 = -1$; $x_2 = (1, 0, 1), l_2 = +1$; $x_3 = (1, 1, 0), l_3 = +1$; $x_4 = (1, 1, 1), l_4 = +1$

w_i	Pattern (misclassified)	$w^T x$	Update $\rightarrow w_{(i+1)}$
$w_0 = (0,0,0)$	(1,0,0)	0	$w_1 = (0,0,0) + (-1)(1,0,0) = (-1,0,0)$
$w_1 = (-1,0,0)$	(1,0,1)	-1 ✓	No update
$w_1 = (-1,0,0)$	(1,1,0)	-1 ✓	No update
$w_1 = (-1,0,0)$	(1,0,0)	-1 ✓	No update
...	...	...	After 9 updates
$w_9 = (-1,2,2)$	ALL	correct	TERMINATE

Final $w_9 = (-1,2,2)$: $b = -1$, $w(1)=2$, $w(2)=2$. Discriminant: $g(x) = 2x(1) + 2x(2) - 1$.

Convergence Theorem: If classes are linearly separable, the Perceptron learning algorithm GUARANTEES convergence in a finite number of steps. The learned w can be expressed as $w = \sum_i \alpha_i x_i$ (linear combination of training patterns).

U4-Q8. Backpropagation Algorithm in MLP (5M)

Introduction (Section 8.6.1)

Backpropagation is a well-known optimization algorithm that learns weight vectors in an MLP using gradient descent to minimize prediction error. It has TWO PASSES:

1. Forward Pass:

Input x propagates through layers to produce output $y^{\{obt\}}$.

2. Backward Pass:

Error at output propagates back to update all weights layer by layer.

Gradient Descent Update Rule

$$w_{\text{updated}} = w_{\text{current}} - \eta \times \nabla E(w_{\text{current}})$$

η = learning rate. ∇E = gradient of error E w.r.t. w . Too small $\eta \rightarrow$ slow convergence. Too large $\eta \rightarrow$ oscillates.

Error Function — Cross-Entropy Loss

$$E = -(y^{\{tar\}} \cdot \log(y^{\{obt\}}) + (1 - y^{\{tar\}}) \cdot \log(1 - y^{\{obt\}}))$$

Forward Pass — Sigmoid Activation

$$\sigma(a) = 1 / (1 + e^{-a}), \quad \sigma'(a) = \sigma(a) (1 - \sigma(a))$$

16. Compute activation at each hidden node h : $a_h = \sum w_{hi} \times \text{input}$

17. Apply sigmoid: $h_{\text{output}} = \sigma(a_h)$

18. Propagate to output: $a_O = \sum w_{Oj} \times h_{\text{output}}$

19. Final output: $y^{\{obt\}} = \sigma(a_O)$

Backward Pass — Weight Updates at Output Layer

Partial derivative of E w.r.t. output weight w_{CO} (C = hidden node, O = output):

$$\begin{aligned}
\partial E / \partial w_{CO} &= (\partial E / \partial y^{\text{obt}}) \times (\partial y^{\text{obt}} / \partial a_O) \times (\partial a_O / \partial w_{CO}) \\
\partial E / \partial y^{\text{obt}} &= (y^{\text{obt}} - y^{\text{tar}}) / [y^{\text{obt}}(1 - y^{\text{obt}})] \\
\partial y^{\text{obt}} / \partial a_O &= \sigma(a_O)(1 - \sigma(a_O)) = y^{\text{obt}}(1 - y^{\text{obt}}) \\
\partial a_O / \partial w_{CO} &= h(1) \quad [\text{output of hidden node C}] \\
\text{Combining: } \partial E / \partial w_{CO} &= (y^{\text{obt}} - y^{\text{tar}}) \times h(1) \\
\text{Update: } w_{CO}^{k+1} &= w_{CO}^k + \eta(y^{\text{obt}} - y^{\text{tar}})h(1)
\end{aligned}$$

Backward Pass Through Hidden Nodes (Chain Rule)

$$\begin{aligned}
\partial E / \partial w_{AC} &= (y^{\text{obt}} - y^{\text{tar}}) \times w_{CO} \times y_C \times (1 - y_C) \times x_A \\
\partial E / \partial w_{AD} &= (y^{\text{obt}} - y^{\text{tar}}) \times w_{DO} \times y_D \times (1 - y_D) \times x_A
\end{aligned}$$

Worked Example — XOR Network (Examples 17–19, Textbook Fig. 8.16)

Network: 2 inputs (A,B) → 2 hidden (C,D) → 1 output (O). Weights: $w_{AC}=1$, $w_{BC}=1$, $w_{AD}=8$, $w_{BD}=8$, $w_{CO}=-30$, $w_{DO}=25$. $\eta=0.1$.

Forward Pass for input (0,0):

$$\begin{aligned}
a_C &= 1 \times 0 + 1 \times 0 = 0 \rightarrow y_C = \sigma(0) = 0.5 \\
a_D &= 8 \times 0 + 8 \times 0 = 0 \rightarrow y_D = \sigma(0) = 0.5 \\
a_O &= 0.5 \times (-30) + 0.5 \times 25 = -2.5 \rightarrow y^{\text{obt}} = \sigma(-2.5) = 0.07586
\end{aligned}$$

Target $y^{\text{tar}}=0$. Error=0.07586.

Table 8.8 — Weight Update Computations for w_{CO} ($\eta=0.1$):

x	y^{tar}	y^{obt}	$y^{\text{obt}} - y^{\text{tar}}$	$h(C)$	Δw_{CO}
(0,0)	0	0.07586	0.07586	0.5	+0.037929
(0,1)	1	0.9552	-0.0448	0.731	-0.032749
(1,0)	1	0.9552	-0.0448	0.731	-0.032749
(1,1)	0	0.194	0.194	0.88	+0.17072

$$\begin{aligned}
w_{CO}^{\text{new}} &= -30 - \eta \times (1/4) \times [0.037929 - 0.032749 - 0.032749 + 0.17072] \\
&= -30 - 0.1 \times (1/4) \times 0.143151 = -30 + 0.003579 \approx -29.9964
\end{aligned}$$

Hidden Layer Update (w_{AC}):

$$\begin{aligned}
\partial E / \partial w_{AC} &= (1/4) [-0.0448 \times (-30) \times 0.731 \times 0.269 + 0.194 \times (-30) \times 0.88 \times 0.12] \approx -0.0876 \\
w_{AC}^{\text{new}} &= 1 - 0.1 \times (-0.0876) \approx 1.00876
\end{aligned}$$

Summary: (1) Initialize weights randomly. (2) Forward pass → get y^{obt} . (3) Compute error. (4) Backward pass → chain rule to get $\partial E / \partial w$ for each weight from output to input. (5) Update $w \leftarrow w - \eta \cdot \partial E / \partial w$. (6) Repeat until convergence. The MLP with backpropagation is a universal function approximator — one hidden layer is sufficient to approximate any continuous function.

U4-Q4. MLP Architecture (Summary) (5M)

Introduction

Multi-Layer Perceptron (MLP) is an ANN made of multiple layers of perceptrons. It learns non-linear input-output mappings that a single perceptron cannot (Section 8.6).

Architecture

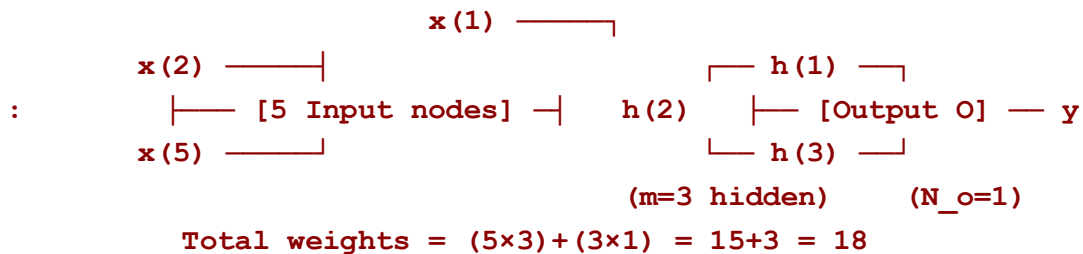
Three Types of Layers:

- **INPUT LAYER:** Receives augmented pattern x_a of size $l+1$ (l features + 1 bias). Each node corresponds to one feature.
- **HIDDEN LAYER(S):** Intermediate neurons ($h(1), h(2), \dots, h(m)$). Non-linear feature learning happens here. Multiple hidden layers are possible.
- **OUTPUT LAYER:** Produces y . Can have N_o output neurons.

Key Architectural Rules (from Textbook):

- No connections among neurons within a layer.
- Typically no direct connections between input and output layers.
- Neurons between adjacent layers are **FULLY CONNECTED**.

Block Diagram (Fig. 8.12 — 5 inputs, 3 hidden, 1 output):



Why Non-linear Activation is Needed

If linear activation is used throughout: $h = A_1 x$, $y = A_2 h = A_2 A_1 x = Bx$. This collapses to a single linear mapping — cannot represent non-linear functions. Sigmoid is used:

$$\sigma(a) = 1 / (1 + e^{(-a)}), \quad \sigma'(a) = \sigma(a) (1 - \sigma(a))$$

Universal Approximation

One hidden layer is sufficient to approximate any continuous function for inputs in a specified range. More hidden layers allow learning hierarchical representations.

Layer	Nodes	Activation	Role
Input	$l+1$	Linear (identity)	Receives features
Hidden	m (tunable)	Sigmoid/ReLU	Non-linear feature extraction
Output	N_o	Sigmoid (classif.) / Linear (regr.)	Produces prediction

UNIT 5 — Clustering

U5-Q5. What is Clustering — Types of Clustering Methods (5M)

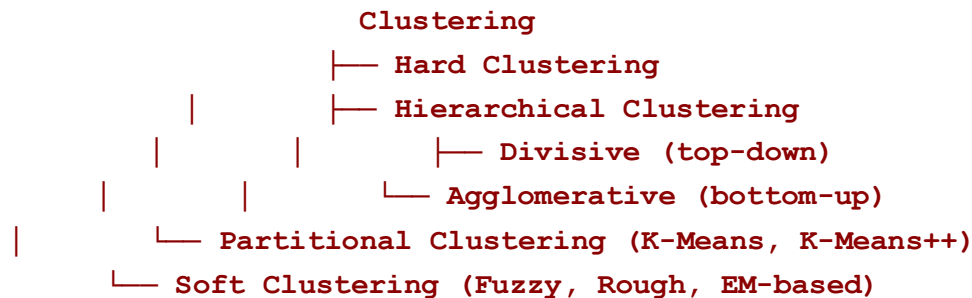
Definition (from Textbook Section 7.1)

Clustering refers to the process of arranging or organizing objects according to specific criteria. It plays a crucial role in uncovering concealed knowledge in large data sets. Clustering involves dividing or grouping the data into smaller data sets based on similarities/dissimilarities.

Formally: let $X = \{x_1, x_2, \dots, x_n\}$ be n data points. Find partition into C clusters $\{X_1, X_2, \dots, X_C\}$ such that:

- $\cup X_i = X$ (every point belongs to some cluster)
- $X_i \neq \emptyset$ for all i
- If $\forall i, j, i \neq j, X_i \cap X_j = \emptyset \rightarrow$ HARD clustering; else $\rightarrow$ SOFT clustering.
- Intra-cluster distance MINIMISED. Inter-cluster distance MAXIMISED.

Classification of Clustering Algorithms (Fig. 7.3)



1. Partitional Clustering (Section 7.5)

Divides data into k non-overlapping clusters. Each point belongs to EXACTLY ONE cluster. Criterion: minimise within-cluster sum of squared errors (IC-SSE).

$$IC-SSE = \sum_i \sum_{x \in C_i} (x - \mu_i)^2$$

- K-Means: Iteratively assign points to nearest centroid, recompute centroids.
- K-Means++: Smart initialisation — selects initial centroids far apart probabilistically.
- Limitation: Sensitive to initial centroids; assumes spherical clusters.

2. Hierarchical Clustering (Section 7.3, 7.4)

Generates a SEQUENCE of data partitions represented as a tree (DENDROGRAM). Two types:

- DIVISIVE (top-down, Section 7.3): Start with all patterns in one cluster, repeatedly split using polythetic (multi-feature) or monothetic (one feature at a time) approach.
- AGGLOMERATIVE (bottom-up, Section 7.4): Start with n singleton clusters, repeatedly merge the closest pair using proximity matrix. Uses single-link, complete-link, or average-link strategy.
- Once two patterns are placed in the same cluster in agglomerative, they remain together in all subsequent levels.

3. Soft Clustering — Fuzzy C-Means (Section 7.6.1)

Each data point can belong to MULTIPLE clusters with a degree of membership $\mu_{ij} \in [0,1]$, where $\sum_j \mu_{ij} = 1$ for each point x_i . Useful when clusters overlap.

4. Rough Clustering (Section 7.7)

Each cluster has a LOWER APPROXIMATION (points definitely in the cluster) and an UPPER APPROXIMATION (points potentially in the cluster). Points in the upper but not lower approximation belong to the boundary/overlap region.

5. EM-Based Clustering (Section 7.8)

Data is modelled as a mixture of probability distributions. EM algorithm iterates between E-step (compute cluster membership probability Z_{ij}) and M-step (update cluster parameters μ_j).

Method	Assignment	Key Parameter	Time Complexity	Best For
K-Means	Hard (one cluster)	k (num clusters)	$O(nlkp)$	Spherical, compact clusters
K-Means++	Hard	k	$O(nlkp)$	Better initialization than K-means
Agglomerative	Hard	None needed	$O(n^2)$	Unknown k, dendrogram needed
Divisive	Hard	None needed	$O(2^n)$ worst	Top-down hierarchy
Fuzzy C-Means	Soft (memberships)	c, M (fuzziness)	$O(nck)$	Overlapping clusters
Rough K-Means	Soft (lower/upper)	k, ϵ , w_l , w_u	Similar to K-means	Boundary cases
EM	Soft (probabilities)	k, σ	$O(nk)$	Gaussian mixture models

U5-Q1. Matrix Factorization in Clustering (5M)

Concept (Section 7.1.5)

It is possible to view clustering as matrix factorization. Let there be n data points in an l -dimensional space represented as matrix $X \in \mathbb{R}^{n \times l}$. It is possible to approximate X as a product of two matrices $B \in \mathbb{R}^{n \times k}$ and $C \in \mathbb{R}^{k \times l}$:

$$X \approx B \times C$$

where B is the CLUSTER ASSIGNMENT MATRIX ($n \times k$) and C is the REPRESENTATIVES MATRIX ($k \times l$) containing cluster centroids. k is the number of clusters.

- HARD clustering: each row of B has exactly one 1 and rest 0s.
- SOFT clustering: rows of B have non-zero entries in $[0,1]$ summing to 1.

Given Data Matrix X

Row	f1	f2	f3
R1	6	5	7
R2	4	6	9
R3	3	5	8
R4	8	8	6
R5	4	6	9

Step 1: Compute Manhattan Distance Matrix

Manhattan Distance: $d(p,q) = \sum |p_i - q_i|$

	R1=[6,5,7]	R2=[4,6,9]	R3=[3,5,8]	R4=[8,8,6]	R5=[4,6,9]
R1=[6,5,7]	0	$ 2 + 1 + 2 =5$	$ 3 + 0 + 1 =4$	$ 2 + 3 + 1 =6$	$ 2 + 1 + 2 =5$
R2=[4,6,9]	5	0	$ 1 + 1 + 1 =3$	$ 4 + 2 + 3 =9$	0
R3=[3,5,8]	4	3	0	$ 5 + 3 + 2 =10$	3
R4=[8,8,6]	6	9	10	0	9
R5=[4,6,9]	5	0	3	9	0

Step 2: Apply Leader Algorithm (Threshold = 3)

20. R1=[6,5,7]: No existing leaders → R1 is LEADER of Cluster 1.
21. R2=[4,6,9]: $d(R2, C1=R1)=5 > 3$ → R2 is LEADER of Cluster 2.
22. R3=[3,5,8]: $d(R3, C1=R1)=4 > 3$; $d(R3, C2=R2)=3 \leq 3$ → R3 assigned to CLUSTER 2.
23. R4=[8,8,6]: $d(R4, C1=R1)=6 > 3$; $d(R4, C2=R2)=9 > 3$ → R4 is LEADER of Cluster 3.
24. R5=[4,6,9]: $d(R5, C1=R1)=5 > 3$; $d(R5, C2=R2)=0 \leq 3$; $d(R5, C3=R4)=9 > 3$ → R5 assigned to CLUSTER 2.

Final Clusters: C1={R1}, C2={R2,R3,R5}, C3={R4}

Step 3: Cluster Assignment Matrix B (5×3)

Row	Cluster	B row
R1	C1	[1, 0, 0]
R2	C2	[0, 1, 0]
R3	C2	[0, 1, 0]
R4	C3	[0, 0, 1]
R5	C2	[0, 1, 0]

Step 4: Representatives Matrix C (3×3)

Using centroid as representative:

$$C1 = [6, 5, 7] \quad (\text{only } R1)$$

$$C2 = \text{centroid of } \{R2, R3, R5\} = [(4+3+4)/3, (6+5+6)/3, (9+8+9)/3] = [11/3, 17/3, 26/3] \approx [3.67, 5.67, 8.67]$$

$$C3 = [8, 8, 6] \quad (\text{only } R4)$$

Matrix Factorization Result: $X \approx B \times C$

$$\begin{bmatrix} 6 & 5 & 7 \\ 4 & 6 & 9 \\ 3 & 5 & 8 \\ 8 & 8 & 6 \\ 4 & 6 & 9 \end{bmatrix} \approx \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} C1 \\ C2 \\ C3 \end{bmatrix} = \begin{bmatrix} 6 & 5 & 7 \\ 3.67 & 5.67 & 8.67 \\ 3.67 & 5.67 & 8.67 \\ 8 & 8 & 6 \\ 3.67 & 5.67 & 8.67 \end{bmatrix}$$

Note: R2 and R5 are identical ([4,6,9]) and are correctly in the same cluster. The approximation is exact for R1 and R4 (leaders). R2, R3, R5 are approximated by C2=[3.67,5.67,8.67]. The matrix factorization compresses $n \times l$ data into $(n \times k + k \times l)$ storage.

U5-Q2. K-Means Clustering — Squared Euclidean Distance (5M)

K-Means Algorithm (Section 7.5.1)

25. Select k initial cluster centres.
26. Assign each point to nearest centre.
27. Recompute centres as mean of assigned points.
28. Repeat 2–3 until no reassignment change.

Given

Points: (7,4),(5,9),(3,3),(1,3),(8,3),(10,1). $k=2$. Initial: C1=(5,9), C2=(8,3).

Squared Euclidean Distance: $d^2(p,q) = (p_1 - q_1)^2 + (p_2 - q_2)^2$

ITERATION 1 — Assignment

Point	Coords	d^2 to C1=(5,9)	d^2 to C2=(8,3)	Assigned
P1	(7,4)	$(7-5)^2 + (4-9)^2 = 4 + 25 = 29$	$(7-8)^2 + (4-3)^2 = 1 + 1 = 2$	C2
P2	(5,9)	$0 + 0 = 0$	$9 + 36 = 45$	C1
P3	(3,3)	$(3-5)^2 + (3-9)^2 = 4 + 36 = 40$	$(3-8)^2 + (3-3)^2 = 25 + 0 = 25$	C2
P4	(1,3)	$(1-5)^2 + (3-9)^2 = 16 + 36 = 52$	$(1-8)^2 + (3-3)^2 = 49 + 0 = 49$	C2
P5	(8,3)	$(8-5)^2 + (3-9)^2 = 9 + 36 = 45$	$(8-8)^2 + (3-3)^2 = 0 + 0 = 0$	C2
P6	(10,1)	$(10-5)^2 + (1-9)^2 = 25 + 64 = 89$	$(10-8)^2 + (1-3)^2 = 4 + 4 = 8$	C2

After Iteration 1: Cluster 1 = {P2=(5,9)}, Cluster 2 = {P1,P3,P4,P5,P6}

Recompute Centroids

New C1 = (5,9) [only P2]

New C2 = ((7+3+1+8+10)/5, (4+3+3+3+1)/5) = (29/5, 14/5) = (5.8, 2.8)

ITERATION 2 — Reassignment with C1=(5,9), C2=(5.8,2.8)

Point	Coords	d^2 to C1=(5,9)	d^2 to C2=(5.8,2.8)	Assigned
P1	(7,4)	$(2)^2 + (-5)^2 = 29$	$(1.2)^2 + (1.2)^2 = 2.88$	C2
P2	(5,9)	0	$(0.8)^2 + (6.2)^2 = 38.98$	C1
P3	(3,3)	$(-2)^2 + (-6)^2 = 40$	$(-2.8)^2 + (0.2)^2 = 7.88$	C2
P4	(1,3)	$(-4)^2 + (-6)^2 = 52$	$(-4.8)^2 + (0.2)^2 = 23.08$	C2

P5	(8,3)	$(3)^2+(-6)^2=45$	$(2.2)^2+(0.2)^2=4.88$	C2
P6	(10,1)	$(5)^2+(-8)^2=89$	$(4.2)^2+(-1.8)^2=20.88$	C2

SAME ASSIGNMENTS as Iteration 1 → Algorithm CONVERGES. Final Result: Cluster 1 = $\{(5,9)\}$, centroid = (5,9) Cluster 2 = $\{(7,4),(3,3),(1,3),(8,3),(10,1)\}$, centroid = (5.8, 2.8)

U5-Q3. Agglomerative vs Divisive Hierarchical Clustering (5M)

Hierarchical Clustering Overview

Hierarchical algorithms generate a SEQUENCE of data partitions using a tree structure called a DENDROGRAM. There are two types (Section 7.3, 7.4):

Aspect	Agglomerative (AGNES)	Divisive (DIANA)
Approach	Bottom-up	Top-down
Starting point	n singleton clusters (each point = cluster)	1 cluster containing all n points
Process	Iteratively MERGE closest pair of clusters	Iteratively SPLIT cluster with max variance
Stopping	When all points are in one cluster (or desired k)	When all clusters are singletons (or desired k)
Complexity	$O(n^2)$ — compute all pairwise distances	$O(2^n)$ polythetic; $O(n \log n)$ monothetic
Linkage strategy	Single-link, Complete-link, Average-link	Max variance split, monothetic/polythetic
Direction	Merging is irreversible (once merged, stays merged)	Splitting is irreversible
Dendrogram	Built from leaves to root	Built from root to leaves

Agglomerative Algorithm (Example 13 — Table 7.12, 8 data points)

Data points: x_1 (5.5,5.5), x_2 (7,6.5), x_3 (7,5.5), x_4 (10,6), x_5 (10.75,6), x_6 (10,8), x_7 (10.5,8), x_8 (7,8)

Using city-block (Manhattan) distance. From Table 7.13: minimum distance = 0.5 between $\{x_6, x_7\}$ and also $\{x_4, x_5\}$.

29. MERGE x_6 and x_7 ($d=0.5$) → $C1=\{x_6, x_7\}$. 7 clusters remain.

30. Next minimum: x_4, x_5 ($d=0.75$) → MERGE → $C2=\{x_4, x_5\}$. 6 clusters remain.

31. x_2 and x_3 have $d=1.0$ → MERGE → $C3=\{x_2, x_3\}$. 5 clusters remain.

32. $C3$ and x_1 merge ($d=1.5$) → $C4=\{x_1, x_2, x_3\}$. 4 clusters.

33. $C4$ and x_8 merge → $C5=\{x_1, x_2, x_3, x_8\}$. $C2$ and $C1$ merge → $C6=\{x_4, x_5, x_6, x_7\}$.

34. $C5$ and $C6$ merge → Final single cluster.

Dendrogram (Fig. 7.8) shows this hierarchy: $C1, C2, C3, C4, C5, C6$ at successive levels.

Divisive Algorithm (Example 12 — Monothetic, Table 7.12)

Same 8 data points. Feature f1 sorted: $x_1 : 5.5, x_2 : 7, x_3 : 7, x_8 : 7, x_4 : 10, x_6 : 10, x_7 : 10.5, x_5 : 10.75$. Largest gap in f1 = 3 units (between $x_8 : 7$ and $x_4 : 10$). Split at midpoint 8.5:

- $C1 = \{x_1, x_2, x_3, x_8\}, C2 = \{x_4, x_5, x_6, x_7\}$

Split C1 by f2: $x_1 : 5.5, x_3 : 5.5, x_2 : 6.5, x_8 : 8$. Largest gap=1.5 (between x_2 and x_8). Split at 7.25:

- $C11 = \{x_1, x_2, x_3\} = C112 + C111, C12 = \{x_8\}$

Split C2 by f2: x_6, x_7 (f2=8) vs x_4, x_5 (f2=6). Split at 7:

- $C21 = \{x_6, x_7\}, C22 = \{x_4, x_5\}$

Final clusters at this level: $\{x_1\}, \{x_2, x_3\}, \{x_8\}, \{x_4, x_5\}, \{x_6, x_7\}$ (Fig. 7.7).

U5-Q4. Rough Clustering & Rough K-Means (5M)

Rough Clustering Concept (Section 7.7)

In rough clustering, some data points clearly belong to only one cluster while others may belong to two or more clusters. Each cluster is represented using two sets:

- LOWER APPROXIMATION $R(k)$: points that DEFINITELY belong to cluster k.
- UPPER APPROXIMATION $\bar{R}(k)$: points that POSSIBLY belong to cluster k.
- Points in $\bar{R}(k) - R(k)$: BOUNDARY REGION — may belong to multiple clusters.

$$R(S) = \bigcup G_i \text{ where } G_i \subseteq S \quad (\text{lower approx})$$

$$\bar{R}^-(S) = \bigcup G_i \text{ where } G_i \cap S \neq \emptyset \quad (\text{upper approx})$$

Rough K-Means Algorithm (Section 7.7.1)

Parameters: k (clusters), w_l (lower weight), w_u (upper weight), ϵ (threshold), where $w_l + w_u = 1$.

35. Randomly assign each data point to exactly one lower approximation $R(k)$.

36. Compute centroid C_j based on $R(k)$ and $\bar{R}(k)$:

If $R(k) \neq \emptyset$ and $\bar{R}(k) - R(k) = \emptyset$: $C_j = \sum x_i / |R(k)|$ (only lower approx)

If $R(k) = \emptyset$ and $\bar{R}(k) - R(k) \neq \emptyset$: $C_j = \sum x_i / |\bar{R}(k) - R(k)|$ (only boundary)

Else: $C_j = w_l \times \sum x_i / |R(k)| + w_u \times \sum x_i / |\bar{R}(k) - R(k)|$ (weighted combination)

37. Compute Euclidean distance from each data point x to all cluster centres C_j .

38. Use ratio $d(x, C_i) / d(x, C_{\square})$ to determine membership: if ratio $> \epsilon \rightarrow x$ in lower approx $R(k_i)$; else x NOT in lower approx (boundary).

39. Repeat from Step 2 until convergence (Old centroid = New centroid).

Example 17 from Textbook — Data Points

Points: (1,1),(2,2),(4,1),(5,2),(7,0),(8,0). $k=2, w_l=0.7, w_u=0.3, \epsilon=2$.

Step 1: Initial Assignment

$$R(k_1) = \{(1,1), (2,2), (4,1)\}, \quad R(k_2) = \{(5,2), (7,0), (8,0)\}$$

Step 2: Compute Centroids ($R(k) \neq \emptyset$ and $\bar{R}(k) - R(k) = \emptyset$):

$$C1 = ((1+2+4)/3, (1+2+1)/3) = (2.33, 1.33)$$

$$C2 = ((5+7+8)/3, (2+0+0)/3) = (6.67, 0.67)$$

Step 3: Euclidean Distances

Point	$d(x, C1=(2.33, 1.33))$	$d(x, C2=(6.67, 0.67))$	Nearest
(1,1)	$\sqrt{(1.33^2 + 0.33^2)} = 1.37$	$\sqrt{(5.67^2 + 0.33^2)} = 5.68$	C1
(2,2)	$\sqrt{(0.33^2 + 0.67^2)} = 0.75$	$\sqrt{(4.67^2 + 1.33^2)} = 4.86$	C1
(4,1)	$\sqrt{(1.67^2 + 0.33^2)} = 1.70$	$\sqrt{(2.67^2 + 0.33^2)} = 2.69$	C1
(5,2)	$\sqrt{(2.67^2 + 0.67^2)} = 2.75$	$\sqrt{(1.67^2 + 1.33^2)} = 2.14$	C2
(7,0)	$\sqrt{(4.67^2 + 1.33^2)} = 4.86$	$\sqrt{(0.33^2 + 0.67^2)} = 0.75$	C2
(8,0)	$\sqrt{(5.67^2 + 1.33^2)} = 5.82$	$\sqrt{(1.33^2 + 0.67^2)} = 1.49$	C2

Step 4: Apply Ratio Rule (ratio = $d(x, \text{far cluster})/d(x, \text{near cluster})$)

Point	Ratio	Ratio > $\epsilon=2$?	Lower Approx?
(1,1)	$5.68/1.37 = 4.14$	YES	$\rightarrow R(k_1) \checkmark$
(2,2)	$4.86/0.75 = 6.48$	YES	$\rightarrow R(k_1) \checkmark$
(4,1)	$2.69/1.70 = 1.58$	NO	$\rightarrow \text{Boundary}$
(5,2)	$2.75/2.14 = 1.28$	NO	$\rightarrow \text{Boundary}$
(7,0)	$4.86/0.75 = 6.48$	YES	$\rightarrow R(k_2) \checkmark$
(8,0)	$5.82/1.49 = 3.91$	YES	$\rightarrow R(k_2) \checkmark$

Updated Approximations:

$$R(k_1) = \{ (1, 1), (2, 2) \}, \quad R^-(k_1) = \{ (1, 1), (2, 2), (4, 1), (5, 2) \}$$

$$R(k_2) = \{ (7, 0), (8, 0) \}, \quad R^-(k_2) = \{ (4, 1), (5, 2), (7, 0), (8, 0) \}$$

Updated Centroids (both $R(k) \neq \emptyset$ and $\bar{R}(k) - R(k) \neq \emptyset$):

$$\begin{aligned} C1 &= 0.7 \times ((1+2)/2, (1+2)/2) + 0.3 \times ((4+5)/2, (1+2)/2) \\ &= 0.7 \times (1.5, 1.5) + 0.3 \times (4.5, 1.5) = (1.05 + 1.35, 1.05 + 0.45) = (2.4, 1.5) \\ C2 &= 0.7 \times ((7+8)/2, (0+0)/2) + 0.3 \times ((4+5)/2, (1+2)/2) \\ &= 0.7 \times (7.5, 0) + 0.3 \times (4.5, 1.5) = (5.25 + 1.35, 0 + 0.45) = (6.6, 0.45) \end{aligned}$$

Repeat from Step 3 until convergence (new centroids = old centroids). The boundary points (4,1) and (5,2) belong to the upper approximation of BOTH clusters, representing the overlapping region between C1 and C2.

U5-Q6. Agglomerative Clustering — Manhattan Distance (5M)

Given Data

Point	f1	f2
X1	2.5	10

X2	2.5	4.5
X3	7.5	4
X4	4.5	8
X5	7	4.5
X6	6.5	4

Step 1: Compute Manhattan Distance Matrix

$$d(X_i, X_j) = |f1_i - f1_j| + |f2_i - f2_j|$$

	X1	X2	X3	X4	X5	X6
X1	0	5.5	11	4	10	10
X2	5.5	0	5.5	5.5	4.5	4.5
X3	11	5.5	0	7	1	1
X4	4	5.5	7	0	6	6
X5	10	4.5	1	6	0	1
X6	10	4.5	1	6	1	0

Computations: $d(X1, X4) = |2.5 - 4.5| + |10 - 8| = 2 + 2 = 4$; $d(X3, X5) = |7.5 - 7| + |4 - 4.5| = 0.5 + 0.5 = 1$; $d(X3, X6) = |7.5 - 6.5| + |4 - 4| = 1 + 0 = 1$; $d(X5, X6) = |7 - 6.5| + |4.5 - 4| = 0.5 + 0.5 = 1$.

Agglomerative Merging (Single-Link Strategy)

STEP 1: Minimum distance = 1 (ties: X3-X5, X3-X6, X5-X6). Merge X5 and X6 → C1={X5,X6}.

Update distances using single-link (min of pair distances):

$$d(C1, X1) = \min(d(X5, X1), d(X6, X1)) = \min(10, 10) = 10$$

$$d(C1, X2) = \min(4.5, 4.5) = 4.5; d(C1, X3) = \min(1, 1) = 1; d(C1, X4) = \min(6, 6) = 6$$

STEP 2: Minimum = d(C1, X3)=1. Merge X3 and C1 → C2={X3,X5,X6}.

$$d(C2, X1) = \min(11, 10) = 10; d(C2, X2) = \min(5.5, 4.5) = 4.5; d(C2, X4) = \min(7, 6) = 6$$

STEP 3: Minimum = d(X1, X4)=4. Merge X1 and X4 → C3={X1,X4}.

$$d(C3, X2) = \min(5.5, 5.5) = 5.5; d(C3, C2) = \min(10, 6) = 6$$

STEP 4: Minimum = d(X2, C2)=4.5. Merge X2 and C2 → C4={X2,X3,X5,X6}.

$$d(C4, C3) = \min(d(X2, C3), d(C2, C3)) = \min(5.5, 6) = 5.5$$

STEP 5: Minimum = d(C3, C4)=5.5. Merge → Final cluster {X1,X2,X3,X4,X5,X6}.

Summary Dendrogram

Step	Action	Distance	Clusters Formed
1	Merge X5,X6	1.0	{X5,X6}
2	Merge {X5,X6},X3	1.0	{X3,X5,X6}
3	Merge X1,X4	4.0	{X1,X4}
4	Merge {X3,X5,X6},X2	4.5	{X2,X3,X5,X6}

5	Merge {X1,X4},{X2,X3,X5,X6}	5.5	Single cluster
---	--------------------------------	-----	----------------

Reading the dendrogram: To obtain 2 clusters, cut above distance 4.5 $\rightarrow$ $C_A=\{X1,X4\}$, $C_B=\{X2,X3,X5,X6\}$. For 3 clusters, cut above 4.0 $\rightarrow$ $\{X1,X4\}$, $\{X2,X3,X5,X6\}$ splits into $\{X2\}$ + $\{X3,X5,X6\}$. Cluster $\{X3,X5,X6\}$ corresponds to the tight bottom-right group in the scatter plot.

U5-Q7. Fuzzy C-Means Clustering — Membership Update (5M)

Algorithm (Section 7.6.1)

Each data point x_j belongs to cluster C_i with membership value $\mu_{ij} \in [0,1]$ where $\sum_i \mu_{ij} = 1$. M is the fuzzification constant ($M=2$ typically).

Step 1 — Compute membership values:

$$\mu_{ij} = d(x_j, C_i)^{-1/(M-1)} / \sum_i d(x_j, C_i)^{-1/(M-1)}$$

Step 2 — Compute fuzzy cluster centroid:

$$C_i = [\sum_j (\mu_{ij})^M \times x_j] / [\sum_j (\mu_{ij})^M]$$

Step 3 — Repeat until no significant change.

Given Data (Example 16, Table 7.20)

Point	f1	f2
X1	1	1
X2	2	2
X3	4	3
X4	5	3

$c=2$ clusters, $M=2$. Initial membership matrix $\mu^{(0)}$:

	X1	X2	X3	X4
C1	1	1	0	0
C2	0	0	1	1

Iteration 0 $\rightarrow$ Compute Initial Cluster Centres

$$C1(f1) = (1^2 \times 1 + 1^2 \times 2 + 0^2 \times 4 + 0^2 \times 5) / (1^2 + 1^2 + 0^2 + 0^2) = (1+2) / (1+1) = 3/2 = 1.5$$

$$C1(f2) = (1^2 \times 1 + 1^2 \times 2 + 0^2 \times 3 + 0^2 \times 3) / (1+1) = 3/2 = 1.5 \rightarrow C1 = (1.5, 1.5)$$

$$C2(f1) = (0^2 \times 1 + 0^2 \times 2 + 1^2 \times 4 + 1^2 \times 5) / (0+0+1+1) = 9/2 = 4.5$$

$$C2(f2) = (0^2 \times 1 + 0^2 \times 2 + 1^2 \times 3 + 1^2 \times 3) / (0+0+1+1) = 6/2 = 3 \rightarrow C2 = (4.5, 3)$$

Compute Euclidean Distances

$$\begin{aligned}
d(C1, X1) &= \sqrt{(1.5-1)^2 + (1.5-1)^2} = \sqrt{0.25+0.25} = 0.71 \\
d(C1, X2) &= \sqrt{(1.5-2)^2 + (1.5-2)^2} = \sqrt{0.25+0.25} = 0.71 \\
d(C1, X3) &= \sqrt{(1.5-4)^2 + (1.5-3)^2} = \sqrt{6.25+2.25} = 2.92 \\
d(C1, X4) &= \sqrt{(1.5-5)^2 + (1.5-3)^2} = \sqrt{12.25+2.25} = 3.81 \\
d(C2, X1) &= \sqrt{(4.5-1)^2 + (3-1)^2} = \sqrt{12.25+4} = 4.03 \\
d(C2, X2) &= \sqrt{(4.5-2)^2 + (3-2)^2} = \sqrt{6.25+1} = 2.69 \\
d(C2, X3) &= \sqrt{(4.5-4)^2 + (3-3)^2} = \sqrt{0.25} = 0.25 \\
d(C2, X4) &= \sqrt{(4.5-5)^2 + (3-3)^2} = \sqrt{0.25} = 0.25
\end{aligned}$$

Compute Updated Membership Values $\mu^{(1)}$ (M=2, so M-1=1)

Formula for M=2: $\mu_{1j} = (1/d(C1, x_j)) / (1/d(C1, x_j) + 1/d(C2, x_j))$

$$\mu_{11} = (1/0.71) / ((1/0.71) + (1/4.03)) = 1.408 / (1.408 + 0.248) = 1.408 / 1.656 = 0.85$$

$$\mu_{12} = (1/0.71) / ((1/0.71) + (1/2.69)) = 1.408 / (1.408 + 0.372) = 1.408 / 1.780 = 0.79$$

$$\mu_{13} = (1/2.92) / ((1/2.92) + (1/0.25)) = 0.342 / (0.342 + 4.0) = 0.342 / 4.342 = 0.08$$

$$\mu_{14} = (1/3.81) / ((1/3.81) + (1/0.25)) = 0.263 / (0.263 + 4.0) = 0.263 / 4.263 = 0.06$$

$$\mu_{21}=0.15, \mu_{22}=0.21, \mu_{23}=0.92, \mu_{24}=0.94 \quad [\text{complement of } \mu_{1j}]$$

	X1	X2	X3	X4
C1 $\mu^{(1)}$	0.85	0.79	0.08	0.06
C2 $\mu^{(1)}$	0.15	0.21	0.92	0.94

Update Cluster Centres from $\mu^{(1)}$

$$\begin{aligned}
C1(f1) &= (0.85^2 \times 1 + 0.79^2 \times 2 + 0.08^2 \times 4 + 0.06^2 \times 5) / (0.85^2 + 0.79^2 + 0.08^2 + 0.06^2) \\
&= (0.7225 + 1.2482 + 0.0256 + 0.018) / (0.7225 + 0.6241 + 0.0064 + 0.0036) = 2.0143 / 1.3566 \\
&\approx 1.5
\end{aligned}$$

$$C2(f1) \approx 4.4, \quad C2(f2) \approx 2.95 \quad [\text{from textbook Example 16}]$$

Final Result (Table 7.21 from textbook after convergence): X1: $\mu_{C1}=0.85, \mu_{C2}=0.15 \rightarrow$ Allocated to C1 X2: $\mu_{C1}=0.79, \mu_{C2}=0.21 \rightarrow$ Allocated to C1 X3: $\mu_{C1}=0.12, \mu_{C2}=0.88 \rightarrow$ Allocated to C2 X4: $\mu_{C1}=0.14, \mu_{C2}=0.86 \rightarrow$ Allocated to C2 Note: Unlike K-Means, FCM gives partial memberships — X1 belongs 85% to C1 and 15% to C2.

U5-Q8. Expectation Maximization (EM) Algorithm for Clustering (5M)

Introduction (Section 7.8)

EM-based clustering represents clusters as probability distributions. The data set is modelled as a combination (mixture) of k probability distributions where each distribution represents a cluster. The EM algorithm estimates the distribution parameters to best fit the data.

Concept

Let $\{x_1, x_2, \dots, x_n\}$ be the data set. Let Z_{ij} = probability that $x_i \in$ cluster c_j .

EM Algorithm Steps (Section 7.8)

Step 1 — E-Step (Expectation): Compute membership probability:

$$Z_{ij} = \exp[-(x_i - \mu_j)^2 / 2\sigma^2] / \sum_k \exp[-(x_i - \mu_k)^2 / 2\sigma^2]$$

This assigns each data point a probability of belonging to each cluster based on distance from cluster centres.

Step 2 — M-Step (Maximization): Update cluster centres:

$$\mu_j = (\sum_i Z_{ij} x_i) / (\sum_i Z_{ij})$$

Step 3:

Repeat Steps 1 and 2 until there is NO CHANGE in μ values (convergence).

Similarity with K-Means

The E-step is similar to the K-means step of assigning data points to the nearest cluster centre — but instead of hard assignment, EM gives PROBABILISTIC (soft) assignments. The M-step is similar to computing new cluster means.

Example 18 (Textbook) — Data: $x_1 = 1, x_2 = 2, x_3 = 6, x_4 = 7$

$c=2$ clusters, $\sigma=1$. Initial: $\mu_1 = x_1 = 1$ (centre of c_1), $\mu_2 = x_4 = 7$ (centre of c_2).

E-Step — Compute Z_{ij} :

$$\begin{aligned} Z_{11} &= \exp[-\frac{1}{2}(1-1)^2] / [\exp(-\frac{1}{2}(1-1)^2) + \exp(-\frac{1}{2}(1-7)^2)] \\ &= \exp(0) / [\exp(0) + \exp(-18)] = 1 / (1 + e^{-18}) \approx 0.9999 \end{aligned}$$

Data point	Z_{i1} (P for c_1)	Z_{i2} (P for c_2)
$x_1 = 1$	0.9999	0.0001
$x_2 = 2$	0.9999	0.0001
$x_3 = 6$	0.0001	0.9999
$x_4 = 7$	0.0001	0.9999

M-Step — Update μ values:

$$\begin{aligned} \mu_1 &= (0.9999 \times 1 + 0.9999 \times 2 + 0.0001 \times 6 + 0.0001 \times 7) / (0.9999 + 0.9999 + 0.0001 + 0.0001) \\ &\approx (0.9999 + 1.9998 + 0.0006 + 0.0007) / 2.0 \approx 3.0 / 2.0 = 1.5 \\ \mu_2 &\approx 6.5 \quad [\text{similarly}] \end{aligned}$$

These new μ values (1.5 and 6.5) are the same as computed initially for this data. Algorithm terminates.

Example with $x_5 = 4$ added (Table 7.24)

$x_5 = 4$ is equidistant from both centres $\rightarrow Z_{51} = Z_{52} = 0.5$. This reflects genuine uncertainty about which cluster x_5 belongs to — an advantage over K-means which would force a hard assignment.

Data point	Z_{i1}	Z_{i2}	Interpretation
$x_1 = 1$	0.9999	0.0001	Definitely cluster 1
$x_2 = 2$	0.9996	0.0004	Mostly cluster 1

$x_3 = 6$	0.0004	0.9996	Mostly cluster 2
$x_4 = 7$	0.0001	0.999	Definitely cluster 2
$x_5 = 4$	0.5	0.5	Equal uncertainty — boundary

EM vs K-Means:

- **K-Means:** Hard assignment (each point in exactly 1 cluster). EM: Soft/probabilistic assignment.
- **K-Means:** Minimises sum-of-squared-distances. EM: Maximises likelihood of the mixture model.
- EM is more general but computationally more expensive.
- EM handles boundary points gracefully ($Z=0.5$ indicates equal membership).
- Both converge to local optima — sensitive to initialisation.

— SHORT ANSWER QUESTIONS —

Short Answer Q1–Q10

Q1. Class Conditional Independence

Class-conditional independence is the assumption that all features $f_1, f_2, \dots, f_n$ are mutually independent given the class label C . Formally:

$$P(f_1=v_1, \dots, f_n=v_n | C) = P(f_1=v_1 | C) \times P(f_2=v_2 | C) \times \dots \times P(f_n=v_n | C)$$

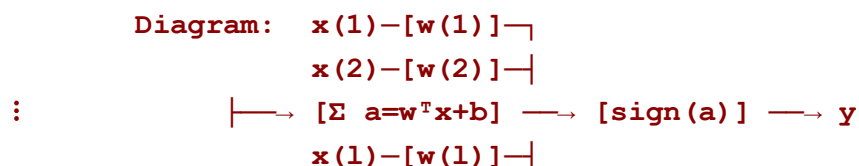
This simplifies computation from $O(\text{exponential})$ to $O(\text{linear in features})$ and forms the basis of the Naïve Bayes Classifier.

Q2. Properties of Random Forest

- Collection of randomly generated DTs; predicts by majority voting (classif.) or averaging (regr.).
- Uses bootstrapping: m samples of size n with replacement, each with $l < L$ random features.
- DTs can be built in parallel — scalable.
- Reduces variance: different data samples reduce DT correlation; random feature subsets simplify each DT.
- Can handle large-dimensional datasets that single DT cannot.
- Model performance improves when DTs work well individually AND have low correlation with each other.

Q3. Structure of Perceptron

The Perceptron has: l input nodes $x(1), \dots, x(l)$ each with weight $w(i)$; a bias node '1' with weight b ; a summation unit computing $a = \sum w(i)x(i) + b$; and a sign activation function outputting $y = \text{sign}(a)$.



$$1 - [b]$$

Q4. SVM Kernel — Definition and Types

A kernel function $K(x_i, x_j)$ computes the inner product of two patterns in a (possibly infinite-dimensional) feature space ϕ without explicitly computing ϕ :

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Popular kernels:

- Polynomial: $K(x_i, x) = (x_i^T x)^p$
- Polynomial up to degree p : $K(x_i, x) = (1 + x_i^T x)^p$
- Gaussian (RBF): $K(x_i, x) = \exp(-||x_i - x||^2 / 2\sigma^2)$ — maps to ∞ -dimensional space
- Sigmoidal: $K(x_i, x) = \tanh(ax_i^T x + b)$

Q5. Linear Discriminant Function — Definition and Example

A Linear Discriminant Function (LDF) separates classes using a hyperplane in the feature space:

$$g(w, x) = w^T x + b = \sum_i w(i)x(i) + b$$

Decision rule: assign x to positive class if $g(w, x) > 0$, negative class if $g(w, x) < 0$.

Example: For Boolean AND gate,

$$g(x) = 3A + 2B - 4$$

Classifies $(1, 1) \rightarrow +1$ (AND=1) and all others $\rightarrow -1$ (AND=0). Decision boundary: $3A + 2B = 4$.

Q6. Weight Update in Perceptron Learning Algorithm

The Perceptron updates weights ONLY when a training pattern is misclassified:

x_j is misclassified by w_i if: $l_j \times (w_i^T x_j) \leq 0$

Update formulas:

If $l_j = +1$ (positive misclassified): $w_{i+1} = w_i + x_j$

If $l_j = -1$ (negative misclassified): $w_{i+1} = w_i - x_j$

General form: $w_{i+1} = w_i + l_j \times x_j$

This moves w in the direction that corrects the misclassification. If no misclassification for n successive patterns $\rightarrow$ TERMINATE.

Q7. Manhattan Distance Matrix for $x1=(4,1)$, $x2=(5,1)$, $x3=(6,3)$

$$d(x1, x2) = |4-5| + |1-1| = 1+0 = 1$$

$$d(x1, x3) = |4-6| + |1-3| = 2+2 = 4$$

$$d(x2, x3) = |5-6| + |1-3| = 1+2 = 3$$

	$x1=(4,1)$	$x2=(5,1)$	$x3=(6,3)$
$x1=(4,1)$	0	1	4
$x2=(5,1)$	1	0	3
$x3=(6,3)$	4	3	0

Q8. Fuzzy Cluster Centroid Formula

The fuzzy cluster centroid C_i for cluster i in Fuzzy C-Means is computed as the weighted mean of all data points, with weights being the M -th power of membership values:

$$C_i = [\sum_j (\mu_{ij})^M \times x_j] / [\sum_j (\mu_{ij})^M]$$

where μ_{ij} = membership value of data point x_j in cluster i ; M = fuzzification constant ($M > 1$, typically 2); x_j = feature vector of data point j .

As $M \rightarrow 1$: approaches hard K-means. As $M \rightarrow \infty$: all $\mu_{ij} \rightarrow 1/c$ (equal membership).

Q9. Centroid and Medoid for $C1=\{(1,1),(1,3),(2,2),(3,1),(3,3)\}$

Centroid (mean of all points):

$$\text{centroid_f1} = (1+1+2+3+3)/5 = 10/5 = 2$$

$$\text{centroid_f2} = (1+3+2+1+3)/5 = 10/5 = 2$$

$$\text{CENTROID} = (2, 2) \quad [\text{Note: this is actually one of the data points!}]$$

Medoid (actual data point with minimum sum of distances to all others):

Compute sum of Euclidean distances for each point:

(2,2): d to (1,1) $=\sqrt{2}=1.41$; d to (1,3) $=\sqrt{2}=1.41$; d to (3,1) $=\sqrt{2}=1.41$; d to (3,3) $=\sqrt{2}=1.41 \rightarrow \text{Sum}=5.64$

(1,1): d to (1,3)=2; d to (2,2)=1.41; d to (3,1)=2; d to (3,3)=2.83 $\rightarrow \text{Sum} \approx 8.24$

(1,3): similar to (1,1) $\rightarrow \text{Sum} \approx 8.24$. (3,1) similar $\rightarrow \text{Sum} \approx 8.24$. (3,3) similar $\rightarrow \text{Sum} \approx 8.24$

CENTROID = (2, 2) (mean of cluster — may not be an actual data point in general). MEDOID = (2, 2) (the actual data point with minimum total distance to all others = 5.64). In this case they coincide since (2,2) is both the mean and the actual data point.

Q10. Agglomerative vs Divisive — 2 Key Differences

Aspect	Agglomerative	Divisive
Approach	Bottom-up: starts with n singleton clusters, merges iteratively to form larger clusters.	Top-down: starts with all n patterns in one cluster, splits iteratively into smaller clusters.
Complexity	$O(n^2)$ — need to compute all pairwise distances initially; manageable for moderate n .	$O(2^n)$ for polythetic (all possible 2-partitions); $O(n! \log n)$ for monothetic — can be expensive.